DESARROLLO WEB BACKEND

UNIDAD 2

Tecnologías Base para el Desarrollo Web

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

2.1 Introducción a HTML5
- 2.1.1 Estructura semántica y elementos principales
- 2.1.2 Formularios y validación

2.2 CSS3
- 2.2.1 Selectores y propiedades
- 2.2.2 Layouts: Flexbox y Grid
- 2.2.3 Diseño responsivo

2.3 JavaScript
- 2.3.1 Sintaxis y estructuras de control
- 2.3.2 Manipulación del DOM
- 2.3.3 Eventos e interactividad

2.4 Visión general de tecnologías y frameworks Backend
- 2.4.1 PHP y Laravel
- 2.4.2 Node.js y Express
- 2.4.3 Python y Django
- 2.4.4 Criterios de selección de tecnología

2.5 Lenguajes de programación del lado del servidor
- 2.5.1 Características y casos de uso de cada lenguaje
- 2.5.2 Comparativa entre lenguajes backend

COMPETENCIA DE LA UNIDAD

Aplicar las tecnologías base del desarrollo web, integrando HTML5, CSS3 y JavaScript con lenguajes del lado del servidor.

2.1 Introducción a HTML5

HTML5 es la quinta y última versión principal del lenguaje de marcado HTML (HyperText Markup Language), estandarizado por el W3C y el WHATWG. Representa una evolución fundamental respecto a versiones anteriores, introduciendo semántica mejorada, APIs nativas para multimedia, almacenamiento local, geolocalización y formularios enriquecidos, eliminando la dependencia de plugins externos como Flash o Silverlight.

> **¿Qué es HTML?**
>
> HTML (HyperText Markup Language) es el lenguaje estándar para crear y estructurar el contenido de páginas web.
> Define el significado y la estructura del contenido mediante elementos representados por etiquetas (<tag>).
> HTML5 fue oficialmente recomendado por el W3C en octubre de 2014.

2.1.1 Estructura semántica y elementos principales

La semántica en HTML5 se refiere al significado preciso que las etiquetas confieren al contenido. A diferencia de HTML4 donde predominaba el uso indiscriminado de e , HTML5 introduce elementos que describen con claridad el propósito de cada sección del documento.

Estructura básica de un documento HTML5

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Aplicación Web</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <header>
      <nav> ... </nav>
    </header>
    <main>
      <article>
        <section> ... </section>
      </article>
      <aside> ... </aside>
    </main>
    <footer> ... </footer>
    <script src="app.js"></script>
  </body>
</html>
```

Elementos semánticos principales

<header> — Encabezado de página o sección, generalmente contiene logo y navegación.
<nav> — Bloque de enlaces de navegación principal del sitio.
<main> — Contenido principal único de la página (uno por documento).
<article> — Contenido autónomo y reutilizable: post, noticia, comentario.
<section> — Agrupación temática de contenido con encabezado propio.
<aside> — Contenido relacionado pero tangencial: barras laterales, widgets.
<footer> — Pie de página o sección: créditos, enlaces legales.
<figure> / <figcaption> — Figura con su leyenda descriptiva.
<time> — Representa una fecha u hora específica.

La correcta aplicación de elementos semánticos mejora la accesibilidad (lectores de pantalla), favorece el posicionamiento SEO y facilita el mantenimiento del código al comunicar su intención de forma explícita.

Elementos multimedia en HTML5

HTML5 introdujo soporte nativo para audio y video, eliminando la necesidad de Flash. También incorporó el elemento <canvas> para gráficos 2D/3D y <svg> para gráficos vectoriales escalables.

```
<!-- Video nativo -->
<video controls width="640" poster="thumbnail.jpg">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Tu navegador no soporta video HTML5.
</video>

<!-- Audio nativo -->
<audio controls>
  <source src="audio.mp3" type="audio/mpeg">
  <source src="audio.ogg" type="audio/ogg">
</audio>

<!-- Canvas 2D -->
<canvas id="miCanvas" width="400" height="200"></canvas>
```

2.1.2 Formularios y validación

HTML5 mejoró significativamente los formularios, incorporando nuevos tipos de input, atributos de validación nativos y elementos especializados que reducen la necesidad de JavaScript para validaciones básicas.

Tipos de input HTML5

Tipo	Descripción	Ejemplo de uso
text	Texto de una línea	Nombre, apellido
email	Dirección de correo con validación	usuario@dominio.com

password	Texto enmascarado	Contraseñas
number	Valor numérico con min/max/step	Cantidad, precio
date	Selector de fecha	Fecha de nacimiento
datetime-local	Fecha y hora local	Reservas, citas
tel	Número de teléfono	+591 70000000
url	URL con validación de formato	https://sitio.com
range	Control deslizante	Volumen, brillo
color	Selector de color nativo	Personalización UI
search	Campo de búsqueda	Barras de búsqueda
file	Selector de archivos	Subida de archivos

```
<form action="/registro" method="POST" novalidate>

  <fieldset>
    <legend>Datos personales</legend>

    <label for="nombre">Nombre completo *</label>
    <input type="text" id="nombre" name="nombre"
           required minlength="3" maxlength="100"
           placeholder="Ej: Juan Pérez">

    <label for="email">Correo electrónico *</label>
    <input type="email" id="email" name="email"
           required autocomplete="email">

    <label for="edad">Edad</label>
    <input type="number" id="edad" name="edad"
           min="18" max="100" step="1">

    <label for="bio">Biografía</label>
    <textarea id="bio" name="bio" rows="4"
              maxlength="500"></textarea>

    <label for="pais">País</label>
    <select id="pais" name="pais" required>
      <option value="">-- Selecciona --</option>
      <option value="BO">Bolivia</option>
      <option value="AR">Argentina</option>
    </select>
  </fieldset>

  <button type="submit">Registrarse</button>
</form>
```

Atributos de validación HTML5

required — El campo no puede estar vacío al enviar el formulario.
pattern='regex' — El valor debe coincidir con la expresión regular indicada.
minlength / maxlength — Longitud mínima y máxima para campos de texto.
min / max — Valor mínimo y máximo para campos numéricos o de fecha.
step — Incremento permitido para campos numéricos o range.
IMPORTANTE: La validación HTML5 es solo del lado cliente. Siempre se debe validar también en el servidor.

2.2 CSS3

CSS3 (Cascading Style Sheets, versión 3) es el lenguaje de estilos estándar para la presentación de documentos HTML. La versión 3 introdujo módulos independientes que permiten animaciones, transiciones, transformaciones, variables nativas (custom properties), nuevos selectores y modelos de layout avanzados como Flexbox y Grid.

> **El modelo de cascada**
> CSS aplica estilos siguiendo un orden de prioridad (cascada):
> 1. Estilos del navegador (user-agent stylesheet) — menor prioridad.
> 2. Estilos del autor (archivos CSS vinculados al documento).
> 3. Estilos en línea (atributo style=").
> 4. Declaraciones !important — mayor prioridad (usar con moderación).
>
> La especificidad del selector también influye: ID > clase > elemento.

2.2.1 Selectores y propiedades

Los selectores CSS determinan a qué elementos HTML se aplicarán los estilos. CSS3 amplió considerablemente el repertorio de selectores, permitiendo reglas muy precisas sin necesidad de agregar clases o IDs adicionales al HTML.

Tipo de selector	Sintaxis	Descripción
Universal	*	Selecciona todos los elementos
De tipo	p, h1, div	Elementos por nombre de etiqueta
De clase	.mi-clase	Elementos con esa clase CSS
De ID	#mi-id	Elemento con ese ID único
Descendiente	div p	p dentro de cualquier div
Hijo directo	ul > li	li hijo directo de ul
Hermano adyacente	h2 + p	p inmediatamente después de h2
Hermano general	h2 ~ p	Todos los p hermanos de h2
Atributo	[type='email']	Elementos con atributo específico
Pseudo-clase	:hover, :focus, :nth-child	Estado o posición del elemento
Pseudo-elemento	::before, ::after	Partes virtuales del elemento

```
/* Variables CSS (custom properties) */
:root {
  --color-primario: #2D5F8A;
  --color-secundario: #0F766E;
  --fuente-base: "Inter", sans-serif;
  --sombra-tarjeta: 0 4px 16px rgba(0,0,0,0.1);
}

/* Selectores avanzados */
form input:not([type="submit"]):focus {
```

```
  border-color: var(--color-primario);
  box-shadow: 0 0 0 3px rgba(45, 95, 138, 0.25);
  outline: none;
}

ul li:nth-child(odd) {
  background-color: #f8fafc;
}

/* Pseudo-elementos */
.seccion-titulo::before {
  content: "";
  display: inline-block;
  width: 4px;
  height: 1em;
  background: var(--color-primario);
  margin-right: 10px;
  vertical-align: middle;
}

/* Transiciones y animaciones */
.boton {
  transition: background-color 0.3s ease, transform 0.2s ease;
}
.boton:hover {
  transform: translateY(-2px);
  background-color: var(--color-secundario);
}
```

2.2.2 Layouts: Flexbox y Grid

Flexbox y CSS Grid son los dos sistemas de layout modernos en CSS3. Flexbox está optimizado para distribución de elementos en una dimensión (fila o columna), mientras que Grid permite diseños bidimensionales (filas y columnas simultáneamente).

> **Flexbox — Conceptos clave**
>
> Flex Container: elemento padre con display: flex o display: inline-flex.
> Flex Items: hijos directos del contenedor flex.
> Eje principal (main axis): dirección definida por flex-direction (row | column).
> Eje cruzado (cross axis): perpendicular al eje principal.
> justify-content: alineación de items en el eje principal.
> align-items: alineación de items en el eje cruzado.
> flex-wrap: permite que los items salten a una nueva línea si no caben.

```
/* ── Flexbox ──────────────────────────────────── */
.navbar {
  display: flex;
```

```
  justify-content: space-between;  /* espacio entre items */
  align-items: center;             /* centrado vertical */
  gap: 1rem;
  padding: 0 2rem;
}

.tarjetas {
  display: flex;
  flex-wrap: wrap;
  gap: 1.5rem;
}

.tarjeta {
  flex: 1 1 280px;  /* crecer | encoger | base */
  min-width: 0;
}

/* ── CSS Grid ──────────────────────────────── */
.dashboard {
  display: grid;
  grid-template-columns: 240px 1fr 320px;
  grid-template-rows: 60px 1fr 40px;
  grid-template-areas:
    "header  header  header"
    "sidebar main    aside"
    "footer  footer  footer";
  min-height: 100vh;
  gap: 0;
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.aside   { grid-area: aside; }
.footer  { grid-area: footer; }

/* Grid con repetición automática */
.galeria {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
  gap: 1rem;
}
```

2.2.3 Diseño responsivo

El diseño responsivo (Responsive Web Design, RWD) es la práctica de crear interfaces que se adaptan automáticamente a cualquier tamaño de pantalla y dispositivo. Se basa en tres pilares: cuadrículas fluidas, imágenes flexibles y media queries.

> **Estrategia Mobile-First**
>
> Mobile-first: se diseñan primero los estilos para pantallas pequeñas y luego se agregan media queries para pantallas mayores.
> Es la estrategia recomendada porque la mayoría del tráfico web proviene de dispositivos móviles.
> Los estilos base (sin media query) aplican al móvil; los min-width amplían el diseño.

```
/* ── Breakpoints estándar (Mobile-First) ─── */
/* Base: móvil (< 640px) */
.contenedor {
  width: 100%;
  padding: 1rem;
}

.grid-productos {
  display: grid;
  grid-template-columns: 1fr;        /* 1 columna en móvil */
  gap: 1rem;
}

/* Tablet (≥ 640px) */
@media (min-width: 640px) {
  .grid-productos {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Desktop (≥ 1024px) */
@media (min-width: 1024px) {
  .contenedor {
    max-width: 1200px;
    margin: 0 auto;
    padding: 2rem;
  }
  .grid-productos {
    grid-template-columns: repeat(4, 1fr);
  }
}

/* Pantalla grande (≥ 1280px) */
@media (min-width: 1280px) {
  .grid-productos {
```

```
    grid-template-columns: repeat(5, 1fr);
  }
}

/* Preferencia de esquema de color */
@media (prefers-color-scheme: dark) {
  :root {
    --bg: #0f172a;
    --texto: #e2e8f0;
  }
}
```

2.3 JavaScript

JavaScript (JS) es el lenguaje de programación de alto nivel, interpretado y multiparadigma que permite crear interactividad, manipular el DOM, comunicarse con servidores (AJAX, Fetch API) y en el contexto de este curso —junto con Node.js— ejecutarse también del lado del servidor. Su versión moderna ES6+ (ECMAScript 2015 en adelante) introdujo clases, módulos, arrow functions, promesas, async/await y muchas otras características.

2.3.1 Sintaxis y estructuras de control

JavaScript es un lenguaje de tipado dinámico. Las variables se declaran con let (ámbito de bloque, reasignable), const (ámbito de bloque, no reasignable) o var (ámbito de función, desaconsejado en código moderno).

```
// ── Variables y tipos ──────────────────────────
const PI = 3.14159;
let contador = 0;
let nombre = "Huáscar";
let activo = true;
let datos = null;
let indefinido = undefined;

// Desestructuración
const { nombre: n, edad } = { nombre: "Ana", edad: 22 };
const [primero, segundo, ...resto] = [1, 2, 3, 4, 5];

// Template literals
console.log(`Hola, ${nombre}! Tienes ${edad} años.`);

// ── Estructuras de control ────────────────────────
if (contador > 10) {
  console.log("Mayor que 10");
} else if (contador === 0) {
  console.log("Es cero");
} else {
  console.log("Otro valor");
}

// Operador ternario
const mensaje = activo ? "Activo" : "Inactivo";

// Switch
switch (nombre) {
  case "Ana":
    console.log("Hola Ana"); break;
  default:
    console.log("Usuario desconocido");
}
```

```
// ── Bucles ──────────────────────────────────────
for (let i = 0; i < 5; i++) { /* ... */ }

const frutas = ["manzana", "pera", "uva"];
for (const fruta of frutas) { console.log(fruta); }

frutas.forEach((f, i) => console.log(`${i}: ${f}`));

// ── Funciones ───────────────────────────────────
function saludar(nombre = "Mundo") {
  return `Hola, ${nombre}!`;
}

const sumar = (a, b) => a + b;  // Arrow function

// ── Programación asíncrona ──────────────────────
async function obtenerUsuario(id) {
  try {
    const res = await fetch(`/api/usuarios/${id}`);
    if (!res.ok) throw new Error("Error HTTP: " + res.status);
    const datos = await res.json();
    return datos;
  } catch (error) {
    console.error("Fallo al obtener usuario:", error);
  }
}
```

2.3.2 Manipulación del DOM

El DOM (Document Object Model) es la representación en árbol de los elementos HTML que el navegador construye al cargar una página. JavaScript puede leer y modificar este árbol dinámicamente: agregar, eliminar o modificar elementos, cambiar estilos y atributos.

```
// ── Selección de elementos ──────────────────────
const titulo     = document.querySelector("h1");
const botones    = document.querySelectorAll(".btn");
const formulario = document.getElementById("mi-form");

// ── Modificación de contenido ───────────────────
titulo.textContent = "Nuevo título";          // solo texto
titulo.innerHTML   = "Título <em>nuevo</em>"; // con HTML

// ── Modificación de atributos ───────────────────
const img = document.querySelector("img");
img.setAttribute("alt", "Descripción de imagen");
img.src = "/nueva-imagen.jpg";
```

```
// ── Clases CSS ──────────────────────────────────
titulo.classList.add("resaltado");
titulo.classList.remove("oculto");
titulo.classList.toggle("activo");
titulo.classList.contains("activo"); // true/false

// ── Estilos en línea ────────────────────────────
titulo.style.color = "#2D5F8A";
titulo.style.fontSize = "2rem";

// ── Crear y agregar elementos ───────────────────
const li = document.createElement("li");
li.textContent = "Nuevo elemento";
li.classList.add("item-lista");
document.querySelector("ul").appendChild(li);

// ── Eliminar elementos ──────────────────────────
li.remove();

// ── Recorrido del DOM ───────────────────────────
const padre  = titulo.parentElement;
const hijos  = padre.children;
const sig    = titulo.nextElementSibling;
const ant    = titulo.previousElementSibling;
```

2.3.3 Eventos e interactividad

Los eventos son señales que el navegador emite cuando el usuario interactúa con la página (click, input, submit, etc.) o cuando ocurre algo en el ciclo de vida del documento (DOMContentLoaded, load, resize). JavaScript permite escuchar y responder a estos eventos mediante event listeners.

```
// ── Agregar y remover listeners ─────────────────
function manejarClick(evento) {
  evento.preventDefault();   // evita la acción por defecto
  evento.stopPropagation();  // evita la propagación (bubbling)
  console.log("Elemento:", evento.target);
  console.log("Coordenadas:", evento.clientX, evento.clientY);
}

const btn = document.querySelector("#mi-btn");
btn.addEventListener("click", manejarClick);
btn.removeEventListener("click", manejarClick);

// ── Delegación de eventos ───────────────────────
// Más eficiente que agregar listeners a cada elemento
document.querySelector("ul").addEventListener("click", (e) => {
```

```
  if (e.target.matches("li")) {
    e.target.classList.toggle("seleccionado");
  }
});

// ── Evento DOMContentLoaded ──────────────────────
document.addEventListener("DOMContentLoaded", () => {
  console.log("DOM completamente cargado");
  inicializarApp();
});

// ── Formulario: validación y envío ───────────────
formulario.addEventListener("submit", async (e) => {
  e.preventDefault();
  const formData = new FormData(e.target);
  const datos    = Object.fromEntries(formData.entries());

  const res = await fetch("/api/registro", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(datos),
  });

  if (res.ok) {
    mostrarExito("Registro exitoso!");
  } else {
    mostrarError("Error al registrar");
  }
});

// ── Eventos de teclado ───────────────────────────
document.addEventListener("keydown", (e) => {
  if (e.ctrlKey && e.key === "s") {
    e.preventDefault();
    guardarCambios();
  }
});
```

Evento Personalizado (Custom Event)

JavaScript permite crear y disparar eventos personalizados usando CustomEvent:
const evt = new CustomEvent('productoAgregado', { detail: { id: 5, nombre: 'Libro' } });
document.dispatchEvent(evt);
document.addEventListener('productoAgregado', (e) => actualizarCarrito(e.detail));

2.4 Visión general de tecnologías y frameworks Backend

El desarrollo backend abarca la lógica del servidor, la gestión de bases de datos, la autenticación, la seguridad y la exposición de APIs. Existe un ecosistema amplio de lenguajes y frameworks; cada combinación tiene fortalezas específicas según el tipo de proyecto, el equipo de desarrollo y los requisitos de rendimiento.

2.4.1 PHP y Laravel

PHP (Hypertext Preprocessor) es el lenguaje del lado del servidor más extendido en la historia de la web. Creado por Rasmus Lerdorf en 1994 y refactorizado completamente a partir de PHP 5 y PHP 7, actualmente en su versión 8.x con atributos, fibers y JIT compiler.

Laravel es el framework PHP más popular, creado por Taylor Otwell en 2011. Implementa el patrón MVC (Model-View-Controller) y el ORM Eloquent para interacción con bases de datos, proveyendo un stack completo para aplicaciones web modernas.

```
// ── Laravel: Ruta con controlador ──────────────
// routes/api.php
Route::apiResource("productos", ProductoController::class);
Route::middleware("auth:sanctum")->group(function () {
    Route::post("pedidos", [PedidoController::class, "store"]);
});

// app/Http/Controllers/ProductoController.php
class ProductoController extends Controller {
    public function index() {
        return Producto::with("categoria")
            ->where("activo", true)
            ->paginate(15);
    }
    public function store(StoreProductoRequest $request) {
        $producto = Producto::create($request->validated());
        return response()->json($producto, 201);
    }
}

// app/Models/Producto.php
class Producto extends Model {
    use HasFactory, SoftDeletes;
    protected $fillable = ["nombre","precio","stock","categoria_id"];
    protected $casts = ["precio" => "decimal:2", "activo" => "boolean"];
    public function categoria() {
        return $this->belongsTo(Categoria::class);
    }
}
```

2.4.2 Node.js y Express

Node.js es un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google Chrome. Su arquitectura asíncrona no bloqueante y orientada a eventos lo hace especialmente eficiente para aplicaciones con muchas conexiones concurrentes como APIs REST y aplicaciones en tiempo real.

Express.js es el framework minimalista más utilizado para Node.js. No impone una estructura rígida, lo que permite construir desde simples APIs hasta aplicaciones complejas con arquitecturas personalizadas.

```
// ── Express: API REST básica ──────────────────────
const express = require('express');
const app     = express();

app.use(express.json());

// Middleware de autenticación
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ error: "No autorizado" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch { res.status(403).json({ error: "Token inválido" }); }
};

// Rutas
app.get('/api/productos', async (req, res) => {
  const productos = await Producto.findAll({ where: { activo: true } });
  res.json(productos);
});

app.post('/api/productos', authMiddleware, async (req, res) => {
  const { nombre, precio } = req.body;
  const nuevo = await Producto.create({ nombre, precio });
  res.status(201).json(nuevo);
});

app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

2.4.3 Python y Django

Python es un lenguaje de programación de alto nivel, con énfasis en la legibilidad del código y la productividad del desarrollador. Es el lenguaje más popular en ciencia de datos, inteligencia artificial y scripting, y también tiene un sólido ecosistema para desarrollo web.

Django es un framework web de alto nivel para Python que sigue la filosofía 'batteries included': incluye ORM, autenticación, administrador automático, migraciones, formularios y mucho más de fábrica. Ideal para aplicaciones complejas que requieren rapidez de desarrollo.

```
# ── Django: Modelo y API con DRF ─────────────────
# models.py
from django.db import models

class Producto(models.Model):
    nombre     = models.CharField(max_length=200)
    precio     = models.DecimalField(max_digits=10, decimal_places=2)
    stock      = models.PositiveIntegerField(default=0)
    activo     = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

# serializers.py
from rest_framework import serializers
from .models import Producto

class ProductoSerializer(serializers.ModelSerializer):
    class Meta:
        model  = Producto
        fields = ["id", "nombre", "precio", "stock", "activo"]

# views.py
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAuthenticatedOrReadOnly

class ProductoViewSet(ModelViewSet):
    queryset            = Producto.objects.filter(activo=True)
    serializer_class    = ProductoSerializer
    permission_classes  = [IsAuthenticatedOrReadOnly]

# urls.py
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register(r'productos', ProductoViewSet)
```

2.4.4 Criterios de selección de tecnología

Elegir el stack tecnológico adecuado es una decisión estratégica que impacta en el costo de desarrollo, el rendimiento, la escalabilidad y el mantenimiento a largo plazo. Se deben evaluar múltiples factores:

Factores clave para la selección de tecnología

1. Tipo de aplicación: ¿API REST, tiempo real, CMS, e-commerce, microservicios?
2. Rendimiento: Node.js sobresale en I/O concurrente; Python en procesamiento de datos.
3. Ecosistema y librerías: disponibilidad de paquetes para las necesidades del proyecto.
4. Curva de aprendizaje del equipo: aprovechar el conocimiento existente reduce tiempos.
5. Comunidad y soporte: proyectos activos con documentación extensa minimizan riesgos.
6. Escalabilidad: capacidad para crecer horizontal (más servidores) o verticalmente.
7. Costo de infraestructura: algunos stacks requieren más recursos de servidor.
8. Integración con el ecosistema existente: compatibilidad con otras herramientas del proyecto.

2.5 Lenguajes de programación del lado del servidor

Los lenguajes del lado del servidor (server-side languages) son aquellos que se ejecutan en el servidor web y generan respuestas dinámicas —HTML, JSON, XML— que se envían al cliente. La selección del lenguaje determina el paradigma de programación, las herramientas disponibles y el perfil de rendimiento de la aplicación.

2.5.1 Características y casos de uso de cada lenguaje

PHP 8.x

Paradigma: Multi-paradigma (procedimental, OOP, funcional).
Tipado: Dinámico, con soporte para declaraciones de tipo estrictas (PHP 8).
Fortalezas: Enorme ecosistema web, hosting ubicuo y económico, CMS (WordPress, Drupal), integración nativa con MySQL, documentación extensa.
Casos de uso: Sitios web corporativos, blogs, e-commerce (WooCommerce, Magento), CMS, APIs REST con Laravel.
Frameworks: Laravel, Symfony, CodeIgniter, Slim.
Versión actual: PHP 8.3 con fibers, enum nativo, match expression, named arguments.

Node.js (JavaScript)

Paradigma: Orientado a eventos, asíncrono no bloqueante, funcional y OOP.
Tipado: Dinámico (JS) / Estático opcional con TypeScript.
Fortalezas: Un solo lenguaje frontend+backend, excelente en tiempo real (WebSockets), npm con millones de paquetes, alta concurrencia con bajo consumo de recursos.
Casos de uso: APIs REST y GraphQL, aplicaciones en tiempo real (chat, notificaciones), streaming, serverless functions, microservicios.
Frameworks: Express, NestJS, Fastify, Koa, Hapi.
Contexto del curso: NestJS y Express son los frameworks principales de INF560.

Python

Paradigma: Multi-paradigma (OOP, funcional, procedimental). Énfasis en legibilidad.
Tipado: Dinámico, con type hints estáticos opcionales (PEP 484, mypy).
Fortalezas: Sintaxis limpia y legible, liderazgo en IA/ML (TensorFlow, PyTorch, scikit-learn), scripting y automatización, análisis de datos (pandas, numpy).
Casos de uso: APIs REST, aplicaciones web complejas, plataformas educativas, backends de sistemas de IA, ciencia de datos.
Frameworks: Django, FastAPI, Flask, Tornado.
Versión actual: Python 3.12 con mejoras de rendimiento, f-strings mejorados y mejor gestión de errores.

Java y Spring Boot

Paradigma: Orientado a objetos, fuertemente tipado, compilado a bytecode (JVM).
Tipado: Estático y fuertemente tipado.
Fortalezas: Rendimiento empresarial, robustez, ecosistema maduro (Maven/Gradle), portabilidad (Write Once Run Anywhere), amplio uso en banca y telecomunicaciones.
Casos de uso: Aplicaciones empresariales, microservicios de alto rendimiento, sistemas bancarios, ERPs.

Frameworks: Spring Boot, Quarkus, Micronaut, Jakarta EE.

Ruby y Ruby on Rails

Paradigma: Orientado a objetos puro, con elementos funcionales.
Tipado: Dinámico.
Fortalezas: Convención sobre configuración (CoC), generación rápida de código (scaffolding), herramienta ideal para MVPs y startups, comunidad enfocada en buenas prácticas.
Casos de uso: Startups, MVPs, marketplaces (GitHub, Shopify, Airbnb fueron construidos con Rails), aplicaciones SaaS.
Framework principal: Ruby on Rails 7.x.

Go (Golang)

Paradigma: Procedimental / orientado a objetos ligero, con concurrencia nativa (goroutines).
Tipado: Estático y compilado a binario nativo.
Fortalezas: Rendimiento cercano a C/C++, binarios independientes sin runtime, excelente soporte de concurrencia, bajo consumo de memoria.
Casos de uso: Microservicios de alto rendimiento, herramientas CLI, infraestructura (Docker, Kubernetes están escritos en Go), APIs de alto tráfico.
Frameworks: Gin, Echo, Fiber.

2.5.2 Comparativa entre lenguajes backend

Criterio	PHP/Laravel	Node.js/Express	Python/Django	Java/Spring	Go
Rendimiento	Medio	Alto (I/O)	Medio	Alto	Muy alto
Curva aprendizaje	Baja	Baja-Media	Baja	Alta	Media
Tipado	Dinámico	Din./Estático	Dinámico	Estático	Estático
Concurrencia	Procesos	Event Loop	ASGI (async)	Threads/NIO	Goroutines
Ideal para	Web/CMS	APIs/RT	Web/IA	Enterprise	Microserv.
Hosting/Despliegue	Muy fácil	Fácil	Fácil	Moderado	Fácil
Ecosistema npm/pkg	Composer/Med	npm/Enorme	pip/Grande	Maven/Grande	go mod/Creciendo
Comunidad	Muy grande	Muy grande	Grande	Muy grande	Creciente

Consideración importante para el curso INF560

El curso INF560 Desarrollo Web Backend se enfoca principalmente en:

- PHP y Laravel para desarrollo monolítico
- Node.js con NestJS/Express y TypeScript, ya que estas tecnologías representan el estado del arte en el desarrollo de APIs modernas con JavaScript.

Sin embargo, es fundamental conocer el panorama completo del backend para tomar decisiones tecnológicas informadas en proyectos reales.
Se recomienda comparar las fortalezas de cada stack al momento de diseñar la arquitectura de un sistema.

Resumen de la Unidad 2

Sección	Tema	Conceptos clave
2.1	HTML5	Semántica, multimedia, formularios, validación nativa
2.2	CSS3	Selectores, Flexbox, Grid, diseño responsivo, mobile-first
2.3	JavaScript	ES6+, DOM, eventos, async/await, Fetch API
2.4	Frameworks BE	PHP/Laravel, Node.js/Express, Python/Django, criterios selección
2.5	Lenguajes BE	Características, casos de uso, comparativa multilenguaje